

Relatório de Paralelismo no Knapsack

PPC 2025

Pedro Lopes 58196

Informações prévias:

- Este relatório não segue nenhuma estrutura formal, não contendo índice, bibliografia nem eventuais referências;
- O script para a criação de boxplots e de análise estatística (GraphGen.py) foi criado com inteligência artificial (ChatGPT-5 mini), uma vez que o foco deste trabalho é a criação e análise de algoritmos paralelos, e não a criação de gráficos. A versão final deste script contém poucas alterações quando comparada ao resultado do prompt original.

Resumo:

Foi dada a tarefa de paralelizar um algoritmo genético que pretende resolver o problema da mochila (Knapsack Problem). Segue uma lista de padrões considerados para resolver esta questão, e a justificação para a sua utilização ou descarte:

Fork/Join: Primeira opção considerada e descartada. Devido à simetria das tasks criadas no código e à dificuldade em encontrar um compromisso que evitasse *over-parallelization* e ao mesmo tempo utilizasse eficientemente a capacidade de multithreading do computador, considerei que não seria a melhor opção para este problema. Para problemas com maior irregularidade de tasks, consideraria utilizar esta opção, mas para a situação que estamos a analisar, não penso que seja a mais apropriada.

Map/Reduce: Não foi considerada de forma séria devido a ser mais útil para linguagens funcionais/ programas escritos de forma funcional, o que obrigaria a uma reformulação do código.

Master/Worker: Acabou por ser o método escolhido. Não foi considerado nenhum tipo de WorkStealing, devido à simetria entre workers e entre tasks, além de que os testes feitos (não documentados aqui) com uma WorkStealingThreadPool não apresentaram resultados notáveis para serem considerados. Foram então criados dois algoritmos: Um que utiliza um ExecutorService, através de uma FixedThreadPool, e uma que utiliza a classe Stream do Java.

Análise de Resultados e Metodologia:

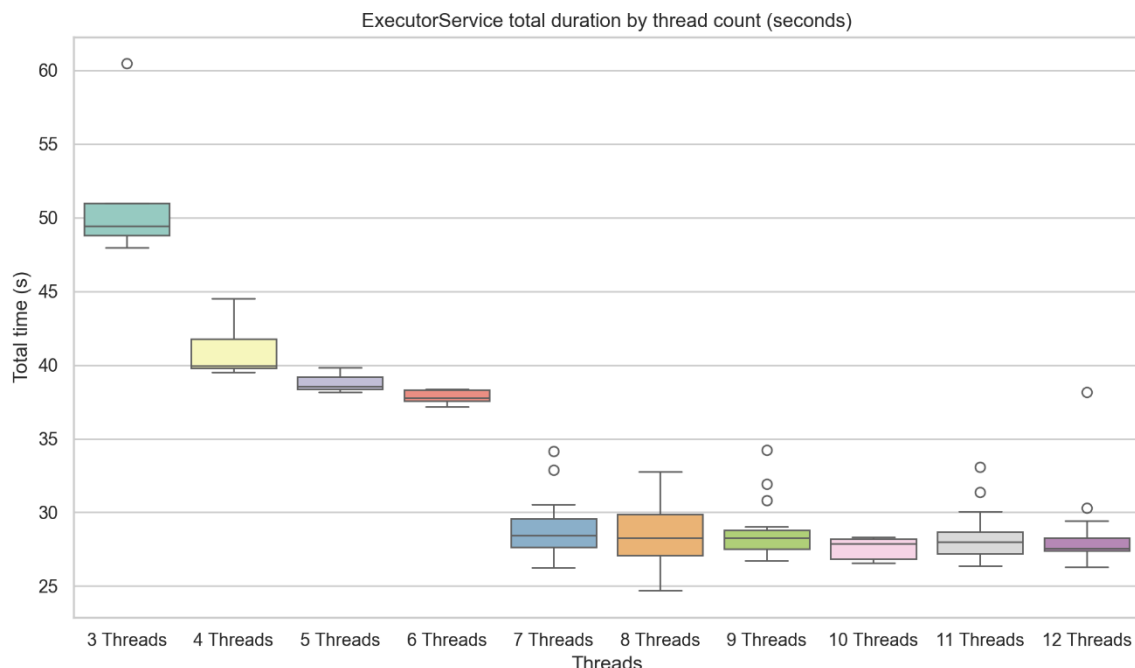
O primeiro passo foi guardar os de execução do algoritmo original e dos seus componentes, e compará-los aos tempos de execução nos dois algoritmos que beneficiavam de paralelismo. Para tal, cada algoritmo foi executado 5 vezes (Em testes futuros, os algoritmos foram executados entre 15 a 20 vezes) Apesar de não ter sido guardada nenhuma amostra do resultado em CSV ou prints, as seguintes conclusões foram tiradas:

- As funções que utilizam paralelismo são consideravelmente mais rápidas que a versão sequencial original;
- A função `BestOf()` não beneficiava de nenhuma diferença considerável de tempo de execução em versões paralelas comparadas à versão original sequencial. Sendo assim, decidi manter a versão original inalterada.

Para a realização destes testes, foi criada uma classe `BootleNeckTester`, que consiste simplesmente na classe `KnapsackGA` com a adição de cálculos de tempo de execução e o arquivo desses cálculos num CSV. Passamos então para o capítulo seguinte, onde se analisou as diferenças no tempo de execução na classe `AlgoritmoExecutor` quando se utiliza diferentes quantidades de testes

Executor Service:

Ao executar o comando `Runtime.getRuntime().availableProcessors()`, o sistema devolvia o número 8. Assim, fazendo $8 - 1$, chegamos ao número 7, que foi a base dos testes da classe `AlgoritmoExecutor`. A partir daí, foram feitos benchmarks com vários números de threads disponibilizadas ao programa, que iam desde 3 a 12. Os resultados foram guardados num CSV chamado `Executor_Results.csv`, e compilados através do GraphGen num gráfico que compara o tempo total de execução de cada iteração: Foram realizadas 5 medições com todas as iterações, e 10 medições adicionais com as iterações de 7 a 12 threads



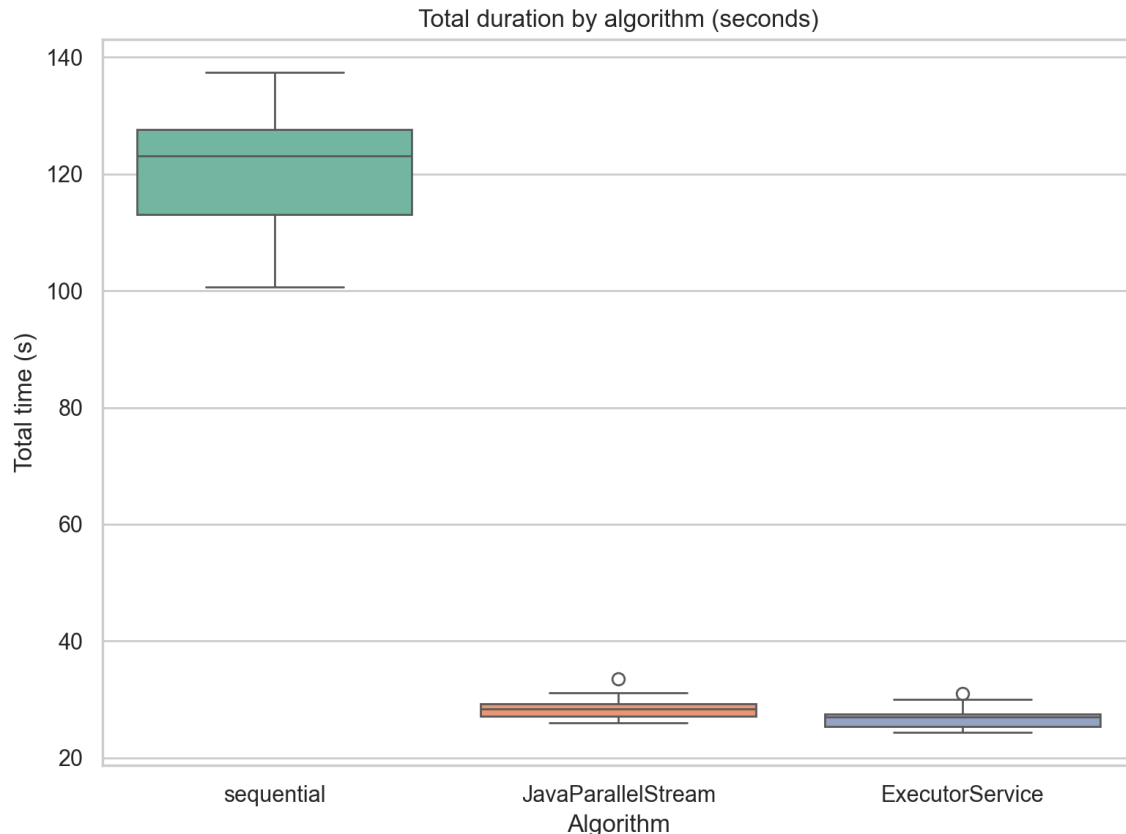
No gráfico, podemos observar que a utilização de menos que 7 threads implica um aumento considerável de tempo de execução quando comparado com as outras instâncias. Podemos também observar que as iterações com utilização de qualquer número de threads acima de 7 não apresentam uma diferença considerável de tempo de execução quando comparadas entre si. Não foram testadas nenhuma iteração com mais de 12 threads, mas considero que o mais provável seria uma deterioração gradual da performance devido a over-parallelization.

Está também disponível o ficheiro stats_result_executor.txt, criado pelo GraphGen, que apresenta a media, mediana, desvio padrão, etc. de cada iteração, além de uma análise estatística com um teste de Mann-Whitney U, que compara cada par de iterações para verificar a sua semelhança.

Sendo assim, decidi avançar com a utilização de 7 threads na classe AlgoritmoExecutor, e passei para a fase final de comparação com o outro algoritmo, AlgoritmoStream.

Comparando resultados:

Passo agora para a comparação entre a classe original KnapsackGA (com os seus tempos medidos pela classe cópia BootleNeckTester), AlgoritmoExecutor e AlgoritmoStream. O algoritmo stream consiste na utilização de Streams e da sua função parallel() para paralelizar vários pontos do algoritmo original. Os resultado depois de 20 execuções de cada classe foram compilados no gráfico seguinte:



Como é possível observar, as duas classes com algoritmos paralelos tiveram um tempo de execução cerca de 4 vezes menor que o algoritmo original. A diferença entre os tempos de execução destas duas classes é de cerca de 4%, que pode ser visto como pouco ou nada insignificante. Além disso, como é possível verificar no ficheiro stats_results.txt, ambas estas classes apresentam um coeficiente U no teste Mann-Whitney U de 93, resultado bastante alto quando comparado a outros pares presentes na entrega. Isto significa que uma amostra (tempo total de execução) de uma destas classes teria uma grande probabilidade de pertencer à outra classe, o que demonstra a similaridade de performance entre estas duas. Mais detalhes sobre as medições de tempo e as análises estatísticas podem ser encontradas entre os ficheiros presentes na entrega.

Conclusão:

Ambos as implementações do padrão Master/Worker cumprem a sua função de acelerar eficientemente o algoritmo (assumo aqui que `Java.Stream.parallel()` use este padrão, mas não consigo comprovar devido à opacidade do método). A utilização de um `ExecutorService` com um número de threads fixas tem uma performance marginalmente melhor e permite maior flexibilidade na maneira como utilizamos as threads, mas é um pouco mais trabalhosa de escrever do que a sua alternativa. Concluo então que ambas as alternativas podem ser soluções válidas para o problema apresentado no início.